



## **University of Engineering and Technology, Lahore**

### **Department of Computer Engineering**

**Course:** Object Oriented Programming (CMPE-221L)

**Lab Title:** Pac-Man Game

**Assignment:** Complex Engineering Problem (CEP)

**Instructor:** Mr. Shahmeer Nawaz

**Semester:** 2nd (Session 2025)

**Submitted By:**

Abdullah Saeed

Roll No: 2025-CYS-51

**Submission Date:** 18-APR-26

---

### **Introduction**

Pac-Man is a classic arcade game in which the player controls a character that navigates through a maze while collecting food and avoiding enemies.

This project implements a simplified Pac-Man game using **Object-Oriented Programming (OOP) in Java**, where multiple classes interact with each other to simulate real-time gameplay.

---

### **Objectives**

- To implement a game using OOP principles

- To design interacting classes and objects
  - To understand game logic and control flow
  - To implement collision detection and scoring
- 

## **Complex Engineering Problem (CEP)**

### **– Depth of Knowledge**

The project demonstrates:

- Encapsulation
  - Aggregation
  - Composition
  - Class-based design
- 

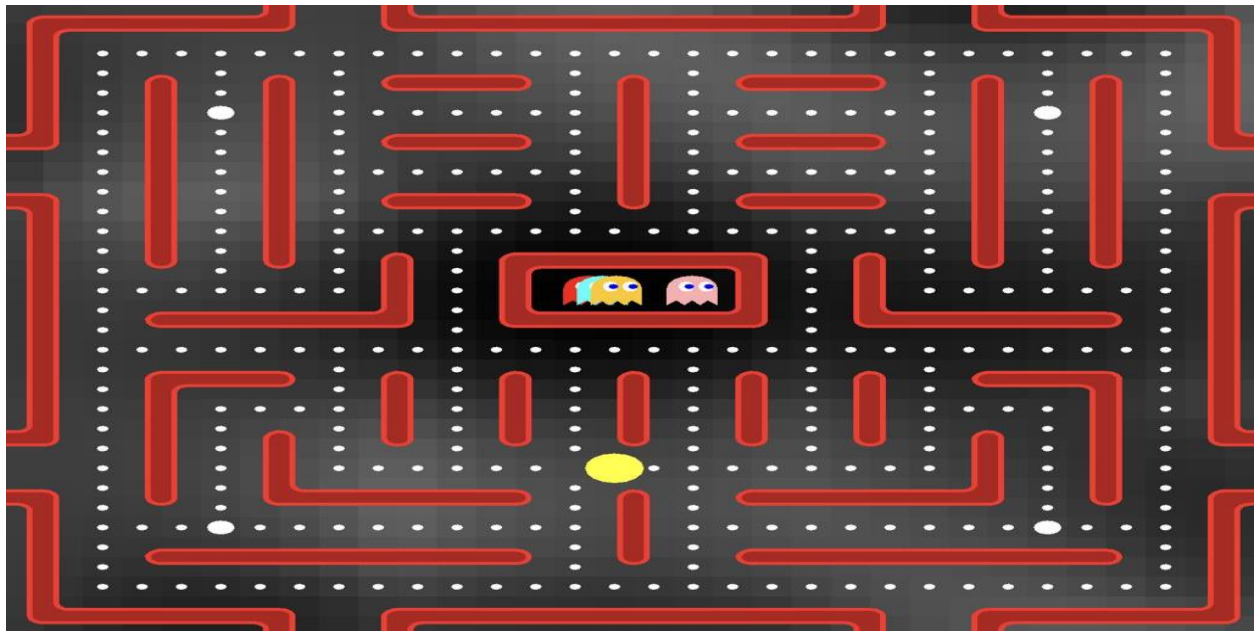
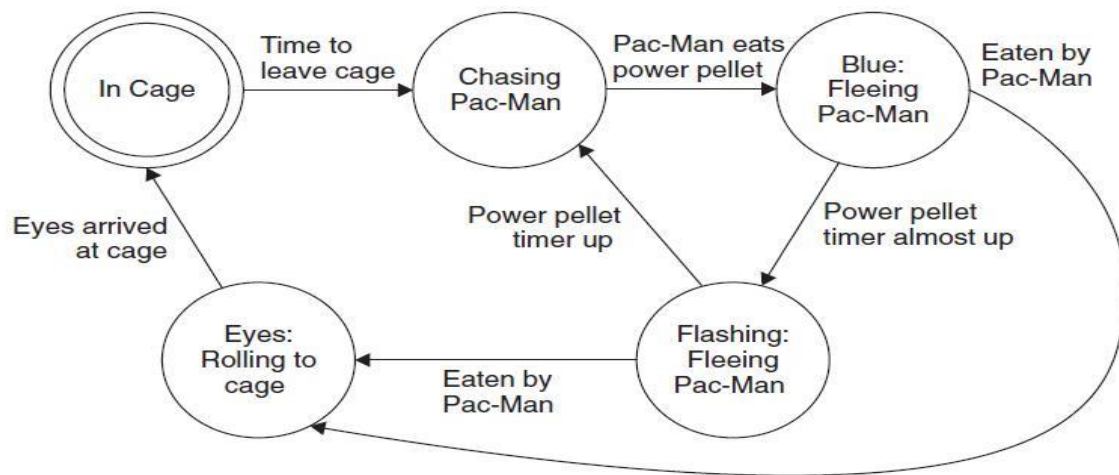
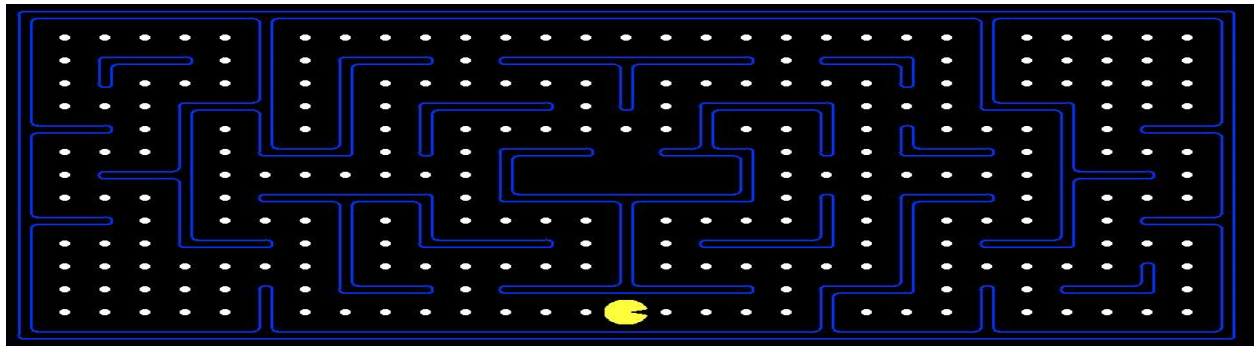
### **– Range of Conflicting Requirements**

- Game complexity vs user simplicity
  - Realism vs playability
  - Difficulty vs user satisfaction
- 

### **– Depth of Analysis**

- Game mechanics and movement
  - Maze (2D array) representation
  - Input handling using Scanner
  - Testing and debugging
-

## System Architecture



## **Components:**

- **PacmanGame** Controls game loop and input
  - **GameBoard** Maze and food system
  - **Pacman** Player
  - **Ghost** Enemy
  - **Food** Food representation
- 

## **Class Design**

### **PacmanGame**

- Main class
- Initializes objects
- Controls game loop

### **GameBoard**

- Stores maze using 2D array
- Handles walls and food

### **Pacman**

- Stores position and score
- Handles movement

### **Ghost**

- Moves randomly
- Acts as enemy

### **Food**

- Represents food items in game

---

## **OOP Concepts Used**

### **Encapsulation**

Data (position, score) is stored inside classes using private variables.

### **Aggregation**

GameBoard contains maze and food elements.

### **Composition**

PacmanGame creates and manages all objects.

### **Abstraction**

Each class performs a specific role.

---

## **Working of the Game**

1. Game starts
  2. Maze is displayed
  3. User inputs movement (W/A/S/D)
  4. Pac-Man moves in maze
  5. Ghost moves randomly
  6. If Pac-Man eats food score increases
  7. If Pac-Man hits ghost game over
- 

## **Output:**

```
Run  PacmanGame x
Move (W/A/S/D): a

Score: 21
# # # # # # # # # #
# .               #
# . # # . # # .   #
# . . # . . # .   #
# # . # # . # #   #
# . P             # #
# G # # # #   # # |
# . . . . #       #
# . . # . . . # . #
# # # # # # # # # #
Move (W/A/S/D): a
Game Over!
Final Score: 22
^
```

---

## **Features**

- Maze with walls
- Player control
- Enemy movement
- Collision detection
- Score system

---

## **Limitations**

- Console-based (no GUI)
- Basic ghost logic
- Single level

---

## **Future Improvements**

- GUI using JavaFX
- Multiple levels
- Smarter ghost AI
- Sound effects

---

## **Conclusion**

This project successfully demonstrates the use of Object-Oriented Programming in building a Pac-Man game. It shows how multiple classes interact to form a complete system and helps in understanding real-world software design.

---